



← Back to student guides

[📈] MLflow

Tracking & registry

3 levels

111 sections



The Complete Guide

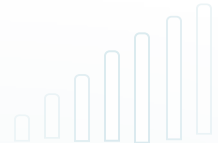
Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

RUN

MODEL

REGISTRY

SERVE



Quick Start

A one-page starter for this tool. Scroll through it here, or open it full screen.

Open PDF





MLOps MENA
COMMUNITY

العربية



Join the Community ↗





18

sections

27

examples

This is part three of three. It closes the series by taking **every topic from Beginner and Mid one level further** (each one now has a security, scale, or ownership dimension) and adds the work you own when MLflow is your team's platform rather than your tool.

Up to this point the questions have been about making a model reproduce and deploy. From here they are different: who can move an alias into production, whether a model from eighteen months ago can still be rebuilt and proven, what the artifact store costs and who pays for it, which component breaks first when the team triples, what happens when the backend database fills up on a Friday night, and where MLflow stops being the right tool.

Where this picks up

Topic from earlier levels	What this level adds
The tracking server	A real deployment: reverse proxy, TLS, workers, and who owns upgrades
Backend store	Database sizing, migrations, index growth, and what breaks first
Artifact store	Proxied access , credential design, lifecycle, and cost
Params, metrics, tags	Retention policy, <code>mlflow gc</code> , and store growth you can predict
Aliases	Promotion as a governed action: approval, separation of duties, audit
Signatures	A published contract with a compatibility policy and consumer migration
Custom pyfunc	Supply-chain risk: pickles, <code>code_paths</code> , and what a model can execute



<code>mlflow.evaluate</code>	Gates that satisfy a reviewer, plus fairness and segment obligations
Datasets	Lineage that survives an audit, and erasure requests
Serving	Authentication, isolation, resource limits, and rollback drills
CI	Credential scoping, fork trust, and who may write to the registry
Debugging	Incident playbooks: lost server, unreadable artifacts, unrebuildable model
new	Auth & multi-tenancy · backup/restore · cost governance · platform ownership · where MLflow ends

I am starting with the trust model, because every other decision in this track depends on it.

The trust model: who can move an alias

The sentence that reframes MLflow security: **moving an alias changes production, and by default nothing stops anyone with tracking credentials from doing it.**

Most teams worry about the inverse and arrive asking "who can see the experiments?" The sharper question is "who can make a model live?"

WRITE PATHS: RANKED BY BLAST RADIUS, NOT BY FREQUENCY

Create runs, log models

Researchers. Harmless: a run is a record

Register a version

Immutable, numbered, linked to its run

Move an alias

This is a production deploy. One call, no review, no record by default



CREDENTIALS, ONE PER PURPOSE

ci-pr

Create runs in `*/ci` only. Cannot register or alias

ci-main

Register and alias, from a reviewed workflow that writes an audit tag

serving

Read the registry and artifacts. No write of any kind

Coarse authorisation

Per-experiment and per-model permissions, not roles, and unauthenticated by default

Clients touch storage directly

Everyone needs bucket credentials unless you enable proxied access

A pickle is executable

Loading a model runs code, in whatever runs your batch job

The dangerous default: an unauthenticated server reachable from the office network, whose aliases drive production serving. Anyone who can reach it can promote any version, including one they logged themselves, with nothing recording who did it. Auth in front, alias writes restricted to a gated job, and an audit tag on every promotion.

Actor	Needs	Must not have
Researcher	Create runs, log models, register versions	Move <code>champion</code> ; delete runs or versions
CI on a pull request	Create runs in a <code>*/ci</code> experiment	Register or alias anything
CI on a protected branch	Register versions; move aliases via a gated job	Delete



Serving / batch consumer	Read the registry and the artifact store	Any write
Retention job	Delete, scoped and audited	Register or alias
A vendor or contractor	Read one experiment	Anything in the registry

Three properties of MLflow's design make this sharper than it looks:

Open-source MLflow has coarse authorisation. The basic auth plugin gives per-experiment and per-registered-model permissions, and that is useful, but it is not rich RBAC, and a plain unauthenticated server gives everyone everything. Say this plainly rather than discovering it in a security review.

Clients touch object storage directly. By default the tracking server returns an artifact URI and the client reads and writes it. That means every user and every CI job needs bucket credentials, which is a much wider credential distribution than people expect, and it is why proxied artifact access matters so much at this level.

A pickle is executable. Loading a model executes code: the unpickling itself, plus any module a custom pyfunc imports. A model artifact from an untrusted source is a code-execution vector in whatever runs your batch job.

The most dangerous default configuration

An unauthenticated tracking server, reachable from the office network, with a registry whose aliases drive production serving. Anyone who can reach it can promote any model version, including one they logged themselves, with no record of who did it. Put auth in front of it, restrict alias changes to a gated job, and make sure the promotion path leaves a trail.

The access shape that works:

separation by credential and by path

Copy

Tracking server (behind TLS + auth)

└─ researchers → create runs, log models, register versions



└ retention → delete, scoped, audited

Artifact store s3://ml-artifacts/

└ prefix per team, write-restricted

└ versioning ON

└ proxied through the tracking server, so clients hold no bucket keys

► TRY IT

1. List every credential that can reach your tracking server: laptops, CI secrets, notebooks, batch jobs, docs. Count them.
2. For each, determine whether it can move an alias. That set is your production-change surface.
3. Check whether clients hold object-store credentials directly, and how many principals that is.
4. Try to answer "who promoted the current champion, and when?" from the data you have today.

Expected result – step four is usually unanswerable, which is the finding. If promotion is a human clicking in a UI, there is no approval trail, and that is the gap to close before anything else on this page.

Authentication, authorisation, and proxied artifacts

Three changes turn a development server into something a team can share.

1. Terminate TLS and require auth at a proxy. MLflow ships basic auth; most teams put OIDC or an existing gateway in front instead.

BASH

Copy

```
# MLflow's built-in basic auth app
mlflow server \
  --backend-store-uri postgresql://mlflow:pass@db/mlflow \
  --artifacts-destination s3://ml-artifacts/mlflow \
  --serve-artifacts \
```



basic_auth.ini

Copy

```
[mlflow]
default_permission = READ
database_uri = postgresql://mlflow:pass@db/mlflow_auth
admin_username = admin
admin_password = <from a secret store>
authorization_function = mlflow.server.auth.authenticate_request_basic_auth
```

granting permissions through the auth API

Copy

```
from mlflow.server import get_app_client

auth = get_app_client("basic-auth", tracking_uri="https://mlflow.internal")
auth.create_user(username="amina", password="...")
auth.create_experiment_permission(experiment_id="12", username="amina", permission="EDIT")
auth.create_registered_model_permission(name="churn-classifier", username="amina", permi
```

Permission	Allows
READ	View runs, metrics, models
EDIT	Create and modify runs and versions
MANAGE	Change permissions, delete
NO_PERMISSIONS	Nothing

2. Turn on proxied artifact access. `--serve-artifacts` with `--artifacts-destination` makes the tracking server broker every artifact transfer, so clients need no bucket credentials at all.

✓ **Proxied artifacts**

- Only the server holds bucket credentials

✗ **Direct client access (the default)**

- Every user and CI job needs bucket credentials



- Clients work with only a tracking token

- Faster for large artifacts, and simpler to scale

3. Restrict who can move an alias. Nothing in MLflow separates "register a version" from "promote it" beyond registered-model permissions, so the separation has to be operational: researchers get `EDIT` on the experiment and `READ` on the registered model; only a CI service account with `EDIT` on the model can move aliases, and it does so from a workflow that requires review.

Proxied artifacts move the bottleneck onto the tracking server

Every model upload and download now flows through it, so a team pushing multi-gigabyte artifacts turns the server into a throughput problem and a single point of failure. It is usually the right trade for the credential story, but size the server for it, and consider direct access for large artifacts with tight bucket policies instead.

► TRY IT

1. Stand up a server with basic auth and confirm an unauthenticated client is rejected.
2. Grant a user `READ` on one experiment and confirm they cannot write to it.
3. Enable `--serve-artifacts` and log a model from a client with no bucket credentials at all.
4. Upload a 2 GB artifact through the proxy and time it. Compare with direct access.

Expected result – step three is the one that changes your credential inventory: a client that needs only a tracking token is a much smaller blast radius. Step four is the cost of that, measured rather than assumed.

Self-hosting: what you own

Beginner ran `mlflow server` on a laptop. A shared deployment has four moving parts and each has its own failure mode.



Component	Holds	If it dies	Grows with
Reverse proxy	Nothing	No access; the store is intact	-
Tracking server	Nothing: stateless	Restart or scale out horizontally	-
Backend database	Runs, params, metrics, tags, registry	Everything stops. This is the system of record	Metric points × runs
Artifact store	Models, plots, reports	Models unreadable; metadata survives	Artifact volume × retention

a production-shaped invocation

Copy

```
mlflow server \  
  --backend-store-uri postgresql://mlflow:${PGPASS}@db.internal:5432/mlflow \  
  --artifacts-destination s3://ml-artifacts/mlflow \  
  --serve-artifacts \  
  --app-name basic-auth \  
  --host 127.0.0.1 --port 5000 \  
  --workers 4 \  
  --gunicorn-opts "--timeout 120 --graceful-timeout 30 --max-requests 2000"
```

docker-compose.yml: the parts that matter

Copy

```
services:  
  db:  
    image: postgres:16  
    environment: { POSTGRES_DB: mlflow, POSTGRES_USER: mlflow }  
    volumes: ['/opt/mlflow/pgdata:/var/lib/postgresql/data']  
    command: ["postgres", "-c", "max_connections=200", "-c", "shared_buffers=1GB"]  
  
  mlflow:  
    image: ghcr.io/mlflow/mlflow:v2.16.0          # pinned, never :latest  
    depends_on: [db]  
    environment:
```



```
mlflow server --backend-store-uri postgresql://mlflow:pass@db/mlflow
--artifacts-destination s3://ml-artifacts/mlflow
--serve-artifacts --host 0.0.0.0 --port 5000 --workers 4
```

Non-negotiables for a server anyone else depends on:

Requirement	Why
TLS at a proxy	Credentials and payloads cross the network otherwise
Postgres or MySQL, not SQLite	SQLite locks under concurrency and cannot be scaled or replicated
Object storage, not a server disk	The file server has no lifecycle, replication, or versioning
Bucket versioning on	Makes a bad delete or overwrite survivable
A pinned server image tag	<code>latest</code> means an unplanned schema migration on the next restart
Multiple workers, with timeouts	One slow search should not block every client
Backups of the database and the bucket	Metadata without artifacts is a catalogue of missing models
Database and disk alerts	The backend store filling up is the outage that stops all logging

SQLite in a shared deployment is a time bomb

It works beautifully for one person and fails under concurrent writers with `database is locked`, at exactly the moment several people are training. Migrating a live SQLite store to Postgres mid-project is unpleasant and involves downtime. Choose Postgres the day a second person needs the server.

► TRY IT



3. Run two concurrent training scripts logging heavily and watch database connections and latency.
4. Kill the tracking server mid-run and observe what the client does, and what the run's final status is.

Expected result – step four is worth knowing before it happens: a killed server leaves runs stuck in `RUNNING` with the tail of their metrics lost, and the client's error depends on where it was. Recognising that state saves a confused hour later.

Backup, restore, and upgrades

Having backups is easy. Having restored one is the part that counts.

A complete MLflow backup has **two** parts, and both are required for a usable system:

01 The backend database

`pg_dump` of the MLflow database, and the auth database too, if you use the basic-auth plugin. This is the system of record: runs, metrics, registry, aliases.

02 The artifact bucket

With versioning enabled, plus a replication or lifecycle policy. Without it, a restored database is a catalogue pointing at nothing.

BASH

Copy

```
# 1. Metadata
pg_dump -Fc -h db.internal -U mlflow mlflow > backup/mlflow-$(date +%F).dump
pg_dump -Fc -h db.internal -U mlflow mlflow_auth > backup/mlflow-auth-$(date +%F).dump

# 2. Artifacts – versioning plus cross-region replication is the real backup
aws s3api get-bucket-versioning --bucket ml-artifacts
aws s3api get-bucket-replication --bucket ml-artifacts
```



The restore drill, rehearsed on a separate host:

BASH

Copy

```
pg_restore -h scratch-db -U mlflow -d mlflow --clean backup/mlflow-2026-08-01.dump
mlflow server --backend-store-uri postgresql://mlflow:pass@scratch-db/mlflow \
              --artifacts-destination s3://ml-artifacts/mlflow --serve-artifacts &

# Then verify, in this order:
# 1. Can you log in?
# 2. Does a known run show its metrics?           ← the database restored
# 3. Does the registry list your aliases correctly? ← promotion state survived
# 4. Can you download a known model artifact?     ← bucket + config are right
# 5. Can a client log a new run?                   ← writes work
```

Upgrades run schema migrations, so they follow the same discipline:

Step	Detail
Read the release notes for every version you skip	Migrations are cumulative; some are one-way
Back up the database first	An interrupted migration is why
Run <code>mlflow db upgrade</code> deliberately	Do not let a container restart migrate for you
Pin the target tag	So the change is a decision, not an accident
Upgrade the server before the clients	Newer clients send fields older servers reject
Announce a window	Clients fail on write; runs in flight lose their tail

BASH

Copy

```
mlflow db upgrade postgresql://mlflow:pass@db.internal/mlflow
```



That failure is discovered during the restore, which is the worst possible time. Verify a model download as an explicit restore step, and treat bucket versioning and replication as part of the backup rather than as a storage detail.

► TRY IT

1. Take both backups today. Note the sizes and how long each takes.
2. Restore onto a scratch host and run the five verification steps in order.
3. Specifically confirm that `models:/name@champion` resolves and downloads.
4. Write down your measured RTO, from "the server is gone" to "a client logged a run", and tell your team the number.

Expected result – a real recovery time instead of an assumed one. Step three is the one that fails most often, because artifact configuration is the part nobody thinks of as being part of the database backup.

Multi-tenancy, quota, and organisation

One MLflow server shared by four teams degrades predictably: hundreds of experiments with no naming convention, a registry where nobody knows which models are live, and a backend database growing for reasons nobody can attribute.

What the open-source server gives you:

Control	Mechanism	Limit
Structure	Experiment naming conventions: <code>team/project/purpose</code>	Convention only
Authorisation	Basic-auth plugin: per-experiment and per-model permissions	Coarse; users and permissions, not roles and groups



Storage isolation	An artifact prefix per team, with its own lifecycle	Enforced by bucket policy, not MLflow, unless proxied
Attribution	Mandatory tags: <code>team</code> , <code>cost_centre</code> , <code>origin</code>	Convention, but auditable and searchable
Registry hygiene	Naming convention plus per-model permissions	No folders; the name <code>is</code> the namespace
Hard isolation	A separate server and database per tenant	Real operational duplication

a workable convention for one shared server

Copy

```
experiments:
  risk/churn/research      risk/churn/ci      risk/churn/validation
  fraud/scoring/research  fraud/scoring/ci
  platform/monitoring
registered models:
  risk-churn-classifier    fraud-scoring-xgb
artifacts:
  s3://ml-artifacts/risk/...    lifecycle: 90d → IA, 400d → delete (except tagged keep)
  s3://ml-artifacts/fraud/...
```

platform/mlflow_wrapper.py: where conventions become defaults

Copy

```
import os
import mlflow

def start_run(project: str, purpose: str, name: str, **kwargs):
    team = os.environ["ML_TEAM"]
    mlflow.set_experiment(f"{team}/{project}/{purpose}")
    tags = {
        "team": team,
        "cost_centre": os.environ["ML_COST_CENTRE"],
        "origin": os.environ.get("ML_ORIGIN", "manual"),
        **kwargs.pop("tags", {}),
    }
    return mlflow.start_run(run_name=name, tags=tags, **kwargs)
```



```
kwargs.setdefault("pip_requirements", "requirements-serving.txt")  
return mlflow.pyfunc.log_model(name="model", python_model=model_or_pyfunc, **kwargs)
```

A thin wrapper is the highest-leverage governance you have

Thirty lines that set the experiment path, the mandatory tags, the registered-model name, and the serving requirements turn every convention here into a default. It beats documentation because nobody has to remember anything, and it gives you one place to change when the layout or the tag vocabulary does.

► TRY IT

1. List your experiments sorted by run count and see whether the structure reflects teams or accidents.
2. Try to answer "how much artifact storage did team X use last month" from the data you have.
3. Check whether any researcher can move a production alias today.
4. Write the wrapper and migrate one team's scripts onto it.

Expected result – step two is usually impossible without prefixes and tags, which is exactly the point: attribution has to be designed in before the bill arrives, because it cannot be reconstructed retroactively.

Cost and retention

Three cost centres, and only one of them is usually noticed.

Centre	Driver	Control
Artifact storage	Models per run × runs × retention	Log models deliberately, lifecycle rules, <code>mlflow gc</code>



Backend database	metric points x runs, plus tags and params	Per-epoch logging, retention on CI and sweep runs
Server infrastructure	Concurrency, and proxied artifact throughput	Workers, and a decision about proxying large artifacts

The artifact store is where the surprises live, because a logged model is much larger than it feels:

what is actually filling the bucket

Copy

```
import mlflow
from collections import defaultdict

sizes = defaultdict(float)
for exp in mlflow.search_experiments():
    runs = mlflow.search_runs(experiment_ids=[exp.experiment_id],
                              max_results=50_000, output_format="pandas")
    sizes[exp.name] = len(runs)
for name, count in sorted(sizes.items(), key=lambda kv: -kv[1])[:10]:
    print(f"{count:>7} {name}")
# Pair this with a bucket inventory report for real bytes per prefix.
```

lifecycle rules per prefix

Copy

```
s3://ml-artifacts/<team>/research/    30d → IA, 180d → Glacier, 400d → delete
s3://ml-artifacts/<team>/ci/          14d → delete
s3://ml-artifacts/<team>/registry/    no expiry – a registered model's artifact must liv
```

A lifecycle rule can delete a registered model's artifact

The registry stores a [pointer](#) . Expiring the object behind a registered version leaves an entry that resolves and then fails to download, and it will be discovered by a batch job at 3am. Keep registry-referenced artifacts out of expiry rules, ideally in their own prefix, and verify before enabling any rule.



runs.

platform/retention.py: scheduled, dry-run gated

Copy

```
import os
from datetime import datetime, timedelta
import mlflow
from mlflow import MlflowClient

APPLY = os.environ.get("RETENTION_APPLY") == "1"
EXEMPT = {"keep", "paper", "release"}
RULES = [("risk/churn/ci", 30), ("risk/churn/research", 365)]

client = MlflowClient()
for experiment_name, days in RULES:
    exp = mlflow.get_experiment_by_name(experiment_name)
    cutoff = (datetime.utcnow() - timedelta(days=days)).timestamp() * 1000
    runs = mlflow.search_runs(experiment_ids=[exp.experiment_id],
                              output_format="list", max_results=50_000)
    for run in runs:
        if run.info.end_time and run.info.end_time > cutoff:
            continue
        if set(run.data.tags) & EXEMPT:
            continue
        if run.data.tags.get("registered_model_version"): # never orphan the registr
            continue
        print(("DELETE " if APPLY else "would delete ") + run.info.run_id, run.info.run_
              if APPLY:
            client.delete_run(run.info.run_id)
```

BASH

Copy

```
# Only this actually reclaims space, and it is a server-side operation
mlflow gc --backend-store-uri postgresql://mlflow:pass@db/mlflow --older-than 30d
```

Waste source	Typical share	Fix
A model logged per sweep child	Often the largest	Log models for the winner only



Checkpoints every epoch as artifacts	very large	best + last, and keep them out of the run's model path
CI runs never deleted	Steady database growth	Tagged <code>origin=ci</code> , deleted on a schedule
Per-batch metric logging	Metric-table growth and slow search	Per epoch
<code>log_input_examples=True</code> everywhere	Small but multiplied, and a data-exposure risk	Off by default
Soft-deleted runs, never collected	Nothing reclaimed at all	<code>mlflow gc</code> on a schedule

► TRY IT

1. Get a bucket inventory report and break down bytes by prefix. Identify the largest contributor.
2. Count metric rows in the backend database and work out how many came from per-batch logging.
3. Run the retention script in dry-run mode and read the list, plus what it would free.
4. Check whether any lifecycle rule could expire an artifact a registered version points at.

Expected result – step four is the one that prevents an incident. A rule that deletes the champion's weights produces a failure with no obvious cause, discovered by a scheduled job rather than by a person.

Scale: which component breaks first

Growth arrives in four dimensions, and each breaks something different. Knowing the order lets you fix the right thing.



1 Metric write rate



2 Run count

Hundreds of thousands of runs. `search_runs` and the experiment table get slow.

3 Artifact volume

Storage cost, and, if proxied, tracking-server throughput.

4 Concurrent clients

Worker saturation. Scale the server out; it is stateless.

Dimension	First symptom	Fix, in order
Metric writes	Logging visibly slows the trainer	Per epoch → <code>log_batch</code> → database resources
Run count	Slow table, slow search	Retention → <code>mlflow gc</code> → database indexes and resources
Artifact volume	Bill, or slow proxied transfers	Lifecycle → log fewer models → direct access for huge artifacts
Client concurrency	Timeouts and 502s	More <code>--workers</code> → more replicas behind the proxy
Registry size	Cluttered, ambiguous model list	Naming convention, archive unused models

Two client behaviours cause most server load, and both are code changes rather than infrastructure ones:

PYTHON

Copy

```
# The single worst pattern: a call per batch
for step, batch in enumerate(loader):
    mlflow.log_metric("loss", loss, step=step)           # thousands of HTTP round trips

# Better: per epoch
mlflow.log_metric("loss", epoch_loss, step=epoch)
```



```
points = [Metric("loss", v, int(time.time() * 1000), step=i) for i, v in enumerate(losses)]
client.log_batch(run_id=run_id, metrics=points)
```

PYTHON

Copy

```
# And: do not query in a loop
for name in model_names:
    runs = mlflow.search_runs(filter_string=f"tags.model = '{name}'")    # N expensive qu

# Query once, filter in pandas
frame = mlflow.search_runs(experiment_names=["risk/churn/research"], max_results=20_000)
grouped = frame.groupby("tags.model")
```

The backend database is the single point of failure

MLflow's server is stateless and trivially replaceable; the database is not. When it is full, slow, or locked, logging fails, the UI fails, and the registry, including alias resolution for production serving, fails with it. Monitor its disk and connections as your primary platform alert, and make sure production serving caches the resolved model rather than resolving an alias on every request.

► TRY IT

1. Find your highest-writing run and count its metric points. Compare against a per-epoch equivalent.
2. Count runs per experiment and identify what is filling the database, usually CI and sweeps.
3. Take the database offline for one minute and see what fails, including whether serving keeps working.
4. Add `--workers` and measure concurrent client throughput before and after.

Expected result – step three is the important drill: if your serving path resolves `@champion` on every request, a database blip becomes a production outage. Caching the resolved model at startup is a one-line change that removes that coupling.



MLflow records the commit, the params, and the model's pip dependencies. That is necessary and not sufficient, and the gaps are specific.

Layer	Recorded by MLflow	How to close the gap
Application code	The commit only , not a diff	Commit before real runs; block dirty-tree promotions
Python packages	Inferred pins in the model	An explicit, committed requirements-serving.txt
Python version	In <code>MLmodel</code>	The base image
System libraries, CUDA	No	The serving base image, by digest
Data	Only if you <code>log_input</code>	A dated immutable source, plus the digest and a param
Seeds	Only if you log them	Log and set them; record determinism flags
Feature logic	Only if it lives in the model	A custom pyfunc with <code>code_paths</code>

the pinning pass, in full

Copy

```
import mlflow

with mlflow.start_run(run_name=name, tags={"git_clean": str(is_clean())}) as run:
    if not is_clean():
        raise SystemExit("refusing to log a promotable run from a dirty tree")

    mlflow.log_params({"seed": SEED, "deterministic": True,
                      "data_partition": PARTITION, "base_image": IMAGE_DIGEST})
    mlflow.log_input(dataset, context="training")

    mlflow.pyfunc.log_model(
        name="model", python_model=ChurnModel(),
        code_paths=["src/features.py", "src/model.py"], # feature logic travels
```



The part that makes it a claim rather than a hope is a scheduled rebuild of the current champion:

platform/rebuild_check.py (weekly)

Copy

```
import mlflow
from mlflow import MlflowClient

NAME, TOLERANCE = "risk-churn-classifier", 0.005
client = MlflowClient()
champion = client.get_model_version_by_alias(NAME, "champion")
original = client.get_run(champion.run_id)

# Rebuild from what was recorded: commit, params, data partition.
rebuilt_uri = retrain_from(
    commit=original.data.tags["mlflow.source.git.commit"],
    params=original.data.params,
)

with mlflow.start_run(run_name=f"rebuild check v{champion.version}",
                      tags={"purpose": "rebuild-check"}):
    result = mlflow.evaluate(model=rebuilt_uri, data=load_holdout(),
                             targets="label", model_type="classifier")
    original_auc = float(champion.tags.get("holdout_auc", original.data.metrics["roc_auc"]))
    delta = abs(result.metrics["roc_auc"] - original_auc)
    mlflow.log_metric("rebuild_delta", delta)
    assert delta ≤ TOLERANCE, f"rebuild drifted by {delta:.4f}"
```

MLflow will record an unreproducible run

A dirty tree, an unversioned data path, an unpinned base image: none of these produce a warning, and the run looks as trustworthy as a good one. That is why the checks have to be yours: refuse to register from a dirty tree, require a dataset input, and run a weekly rebuild. The first time that rebuild runs, it will fail, and that failure is the point.

► TRY IT



3. For each layer in the table, note whether that run pins it. Count the gaps.

4. Add the dirty-tree guard and the required dataset input, then repeat.

Expected result – almost always a drift or a failure on the first attempt, plus a short list of unpinned layers. That named list is far more useful to your team than a vague sense that reproducibility could be better.

Governed promotion: approval and separation of duties

Beginner moved an alias. Mid gated it on thresholds. At this level the question is **who decided**, and whether that decision is reviewable.

MLflow has no built-in approval workflow, so the trail has to be constructed. Three mechanisms, together, are enough:

01 Permissions

Researchers get **EDIT** on experiments and **READ** on registered models. Only a CI service account has **EDIT** on the model, so only CI can move an alias.

02 A gated workflow

Promotion runs from a protected branch with required review, so the human approval is a pull-request approval with a named reviewer.

03 Version tags as the audit record

Who, when, on what data, against which baseline, with which thresholds, written onto the model version by the promotion job.

platform/promote.py: the audit record is the point

Copy



```
from mlflow import MlflowClient
from mlflow.models import MetricThreshold

NAME = "risk-churn-classifier"
client = MlflowClient()

candidate = client.get_model_version(NAME, os.environ["VERSION"])
champion = None
try:
    champion = client.get_model_version_by_alias(NAME, "champion")
except mlflow.exceptions.MlflowException:
    pass

run = client.get_run(candidate.run_id)
if run.data.tags.get("git_clean") != "True":
    raise SystemExit("refusing to promote a model trained from a dirty tree")

result = mlflow.evaluate(
    model=f"models:{NAME}/{candidate.version}",
    data=load_frozen_holdout(), targets="label", model_type="classifier",
    baseline_model=(f"models:{NAME}/{champion.version}" if champion else None),
    validation_thresholds={
        "roc_auc": MetricThreshold(threshold=0.85, min_relative_change=0.005,
                                   greater_is_better=True),
        "worst_segment_auc": MetricThreshold(threshold=0.75, greater_is_better=True),
    },
) # raises on failure – the gate is the exception

audit = {
    "approved_by": os.environ["GITHUB_ACTOR"],
    "approved_at": dt.datetime.utcnow().isoformat() + "Z",
    "workflow_run": os.environ["GITHUB_RUN_ID"],
    "holdout": FROZEN_HOLDOUT_URI,
    "baseline_version": champion.version if champion else None,
    "metrics": {k: round(v, 5) for k, v in result.metrics.items()},
}

client.set_model_version_tag(NAME, candidate.version, "promotion_audit", json.dumps(audit))
if champion:
    client.set_registered_model_alias(NAME, "previous", champion.version)
    client.set_registered_model_alias(NAME, "champion", candidate.version)
```



What code produced this model?	The version's run → <code>mlflow.source.git.commit</code> , and <code>git_clean</code>
What data produced it?	The run's dataset input and <code>data_partition</code> param
What configuration?	The run's params, as executed
Was it evaluated, and how?	The <code>promotion_audit</code> tag: holdout URI, baseline, thresholds, metrics
Who approved it, and when?	The same tag, plus the protected-branch review
Can we prove it has not changed?	Registry versions are immutable; the bucket is versioned

A tag written by the same person who trained the model is not separation of duties

If a researcher can both register a version and move the alias, the audit tag records a self-approval. The control that matters is **permissions**: only the CI service account may move aliases, and its workflow requires a review from someone other than the author. The tag is the record; the permission is the control.

► TRY IT

1. Check whether a researcher can move a production alias today. If yes, that is your first fix.
2. Add the audit tag to your promotion script and promote once. Read the tag back.
3. Add the dirty-tree refusal and try to promote a model trained from uncommitted code.
4. Ask someone to reconstruct the last promotion decision from the registry alone, without asking you.

Expected result – step four is the real test. If they can answer what data, what baseline, what thresholds, and who approved it, you have an audit trail. If they have to ask you, you have institutional memory, which is not the same thing.



The audit chain runs model → version → run → code, data, and configuration. It only exists if each link was recorded, and one script proves whether it was.

platform/lineage.py: generated, not narrated

Copy

```
import json
from mlflow import MlflowClient

def lineage(model_name: str, alias: str = "champion") → dict:
    client = MlflowClient()
    version = client.get_model_version_by_alias(model_name, alias)
    run = client.get_run(version.run_id)
    inputs = [
        {"name": d.dataset.name, "source": d.dataset.source,
         "digest": d.dataset.digest, "context": d.tags.get("mlflow.data.context")}
        for d in (run.inputs.dataset_inputs or [])
    ]
    return {
        "model": {"name": model_name, "version": version.version,
                  "alias": alias, "tags": version.tags},
        "run": {"id": run.info.run_id, "user": run.data.tags.get("mlflow.user"),
                "start": run.info.start_time, "status": run.info.status},
        "code": {"commit": run.data.tags.get("mlflow.source.git.commit"),
                  "source": run.data.tags.get("mlflow.source.name"),
                  "clean": run.data.tags.get("git_clean")},
        "data": inputs or "NO DATASET RECORDED",
        "config": run.data.params,
        "promotion": json.loads(version.tags.get("promotion_audit", "{}")),
    }
```

Gap	Symptom in the report	Fix
Dirty tree	<code>clean</code> is <code>False</code> or absent	Refuse to register from a dirty tree
No data lineage	<code>"NO DATASET RECORDED"</code>	Require <code>log_input</code> ; fail the run without it
No approval trail	<code>promotion</code> is empty	Promotion as a gated job that writes the audit tag



Immutable data source	A source URI that can be overwritten	Dated immutable prefixes
Missing evidence	The run was deleted by retention	Exempt registry-referenced runs

Erasure is the hard case, and MLflow's exposure is narrower than a data platform's but real:

Location	May contain personal data	Deletable
Params and tags	Rarely, unless someone logged an identifier	Yes, but params are immutable; delete the run
Metrics	No	-
Artifacts: predictions, reports	Often: scored records with identifiers	Yes, per artifact
<code>input_example</code> inside a model	Yes , if autologged from real data	Re-log the model without it
Traces (GenAI)	Yes: prompts and responses verbatim	Delete traces; decide retention up front
Dataset profiles	Schema and statistics, not rows	Usually fine
Model weights	Learned from the data; sometimes extractable	Effectively no: retraining is the remedy
Database backups	Everything, historically	Only by expiring backups

Two MLflow-specific places personal data hides

`log_input_examples=True` embeds real training rows inside a model artifact you might publish or share. GenAI traces store prompts and responses verbatim, which on production traffic means user messages. Both are default-adjacent settings that create copies nobody remembers during an erasure request. Turn off input examples, and decide trace redaction and retention before enabling tracing on real traffic.



1. Run the lineage report against your current champion and read every field.
2. Count how many of the five gaps in the table your report reveals.
3. Search your artifacts for prediction files containing identifiers, and for embedded input examples.
4. Write down what you would do, and how long it would take, if an erasure request arrived tomorrow.

Expected result – most teams find a complete code chain, a missing data chain, and no approval record. Step three is the uncomfortable one: scored predictions with customer identifiers are common as artifacts, and nobody classifies them as data.

Serving in production, and the supply chain

Beginner served locally. Two things change at this level: the endpoint is a service with the usual obligations, and the model artifact is a supply-chain component.

The service obligations are ordinary and frequently skipped:

Requirement	Why
Authentication at an ingress or gateway	The scoring server authenticates nobody
TLS	Payloads are your users' data
Resource limits and timeouts	One large batch request should not OOM the pod
A pinned base image, by digest	So the environment cannot change under you
Health and readiness probes	/health and a real scoring warm-up
Cached model resolution at startup	So a database blip is not an outage



A refreshed rollback

move champion to previous, restart or let the cache expire

resolve once, not per request

Copy

```
import mlflow

# At startup: resolve the alias to a concrete version and log which one.
client = mlflow.MlflowClient()
version = client.get_model_version_by_alias(MODEL, "champion")
MODEL_URI = f"models://{MODEL}/{version.version}"
MODEL = mlflow.pyfunc.load_model(MODEL_URI)
LOGGER.info("serving %s version %s", MODEL, version.version) # in every log line's con
```

The supply-chain question is the one people miss. **Loading a model executes code.**

01 Unpickling runs code

A pickle is a program. A malicious or tampered artifact executes in whatever process loads it: your batch job, your serving pod.

02 `code\_paths` ships arbitrary modules

Whatever was copied in runs at load time. That is the feature, and it is also the risk.

03 Inferred dependencies install from an index

A serving environment build resolves packages at deploy time unless pinned by hash, which is a dependency-confusion surface.

04 The artifact store is the trust boundary

Anyone who can write to it can replace the bytes behind a registered version.

Control	Effect
Write-restricted artifact prefixes	Only the tracking server or CI can write model paths



Bucket versioning	A replaced object is recoverable and detectable
Proxied artifact access	Clients never hold bucket credentials; access is logged centrally
Pinned base image digests	The system layer cannot change between builds
Hash-pinned serving requirements	Removes the install-time resolution surface
Never load third-party models unreviewed	Treat an external artifact like an unreviewed dependency

A registry version is immutable metadata pointing at mutable bytes

MLflow will not let you change version 7, but nothing in MLflow stops someone with bucket write access from replacing the object version 7 points at. Every consumer would then load different weights with no error and no record. Write-restrict the artifact prefix, enable versioning, and prefer proxied access. That combination is what makes "version 7" mean something.

► TRY IT

1. List who can write to your artifact bucket prefix. Include CI, notebooks, and admin roles.
2. On a test bucket, overwrite the object behind a registered version and load it. Confirm nothing warns you.
3. Check whether your serving path resolves the alias per request or once at startup.
4. Confirm your serving endpoint requires authentication from outside the cluster.

Expected result – step two is the demonstration that changes priorities: an "immutable" version silently serving different weights is more alarming than any diagram, and it takes two minutes to show.

Incident playbooks



01 **Serving keeps working if it cached its model**

Confirm that first. It decides whether this is an outage or an inconvenience.

02 **Stand up the same pinned image**

A different version runs schema migrations on top of an incident.

03 **Point it at the existing database**

The server is stateless; if the database survived, you are minutes away.

04 **Verify in order**

Login → a known run's metrics → alias resolution → a model download → a new run logging.

05 **Reconcile**

Runs interrupted mid-flight are stuck in `RUNNING` ; close them and note the metric tail that was lost.

The backend database is full or corrupted. Everything stops: logging, the UI, and alias resolution. Free space or fail over to a replica, then fix the cause, usually per-batch metric writes or no retention. Prevention: a disk alert and `mlflow gc` on a schedule.

A model version's artifacts are unreadable. Check bucket versioning for a delete marker or an overwrite; check whether a lifecycle rule expired it; check whether the artifact root changed after the run was created. Roll `champion` back to `previous` while you investigate. That is what `previous` is for.

Someone promoted a model that should not be live. Move `champion` to `previous` immediately. Then read the version's `promotion_audit` tag to establish what happened, and fix the permission that allowed it. If a researcher could move the alias, this is an access-control finding rather than a mistake.



Tracking server lost	Same pinned image, same database	Stateless server, tested restore, published RTO
Database full or corrupt	Free space or fail over	Disk alerts, retention, <code>mlflow gc</code> , per-epoch logging
Artifacts unreadable	Check versioning, lifecycle, and the run's artifact URI	Versioning on, registry prefix exempt from expiry
Wrong model promoted	Roll back to <code>previous</code>	Alias permissions restricted to a gated CI job
Serving down after a promotion	Roll back; check the signature diff	Additive-only signatures; validate before promoting
Model cannot be rebuilt	Run the lineage report	Dirty-tree guard, required dataset input, weekly rebuild check

Write these down where the on-call person can find them

Four short runbooks in the repository beat institutional memory, because the person handling the page is frequently not the person who built the platform. Each needs the first command, the verification step, and who to wake.

► TRY IT

1. Stop the tracking server while a batch scoring job is running. Does the job survive?
2. Run the server-loss playbook on a scratch host, timing each step. That is your RTO.
3. Delete an artifact behind a non-production version and practise recovering it from bucket versioning.
4. Have someone who did not build the platform follow one runbook, and note where they get stuck.

Expected result – step one is the most valuable: if scoring jobs resolve the alias per request, an inconvenience becomes an outage. Caching at startup decouples production from the tracking server entirely, and it is a one-line change.



Knowing the boundaries is a senior skill, and naming them is a strong interview signal.

Need	MLflow	Better tool
Executing and scheduling work	Nothing: no agent, no queue, no retries	Airflow, Dagster, Prefect, Argo, or a CI system
Data versioning with real immutability	Lineage annotation only	DVC, LakeFS, Delta/Iceberg time travel
Point-in-time feature serving, online/offline parity	Not its job	Feast, Tecton, or a warehouse-backed store
High-scale, low-latency serving with autoscaling	A scoring server and a container	KServe, Seldon, Triton, or a managed endpoint
Fine-grained RBAC, groups, SSO-mapped roles	Coarse per-experiment and per-model permissions	A managed MLflow, or a gateway plus a separate server per tenant
Production monitoring and drift as a product	Metrics you log yourself	Evidently, a dedicated monitoring platform
Data quality contracts	No opinion	Great Expectations, dbt tests, Soda
Hyperparameter search	No optimiser of its own	Optuna, Ray Tune: then log to MLflow

The honest summary

MLflow is strongest as the **standard for tracking and model packaging**: a widely-supported artifact format, a registry with aliases, and a UI everyone already knows. It is deliberately not an orchestrator, not a data-versioning system, and not a serving platform. That narrowness is why it composes well with whatever you already run, and it means a complete stack always includes other tools, which is worth saying out loud rather than stretching MLflow over everything.



1. For each row above, decide which side your team is on today.
2. Find one place where MLflow is being stretched past its shape, and name the cost.
3. Find one place where a second tool duplicates something MLflow already does well, and name that cost too.

Expected result – most stacks have one of each. Being able to argue both directions (where to lean on MLflow more, and where to stop) is what distinguishes a senior answer from a preference.

Running MLflow as a platform

When MLflow is shared infrastructure, your job changes from using it to operating it. Five obligations.



Publish the numbers

Runs and storage by team, database size, rebuild-check result. Visible without anyone asking.



Ship a wrapper, not a wiki

Experiment path, tags, registered name, serving requirements: all defaults nobody has to remember.



Own the upgrade cadence

A tested window with a database backup and a rollback, not an emergency migration mid-sprint.



Alert on the platform

Database disk and connections, server 5xx, artifact-store errors. Not on users' failed runs.





unreadable, wrong model promoted, all validated by an outsider.

The monitoring set that matters, in priority order and deliberately short:

01

Backend database disk > 80%

When it fills, logging, the UI, and alias resolution all stop. Highest-value alert in the system.

02

Database connections near the limit

Presents as intermittent client failures that look like flaky code.

03

Tracking server 5xx rate

Clients see hangs and failed logging; often a worker or timeout problem.

04

Artifact-store error rate

Credential expiry and policy changes show up here first, as failed model uploads.

05

Rebuild check failing

Reproducibility has regressed and nothing else would have told you.

06

Artifact storage growth versus budget

Weekly, broken down by prefix, before finance notices.

07

Alias changes

Any movement of a production alias, notified to a channel. Cheap, and it makes promotion visible.



Never page the platform team on a user's failed run

A failed training run belongs to its author, who already has the link. Paging on it trains everyone to ignore the channel, and then the alerts that matter (database disk, artifact-store errors) get lost in the noise.

► TRY IT

1. Build the seven alerts, starting with backend database disk.
2. Add a notification on production alias changes and see how often it fires.
3. Publish one weekly report with runs and storage by team and the rebuild result.
4. Ask someone outside the platform team to follow one runbook and note where they get stuck.

Expected result – the alias-change notification in step two is the cheapest governance win available: it makes every production change visible to the team without any process, and it often reveals promotions nobody knew about.

The review checklist



Area	Check
Auth	Is the tracking server authenticated and behind TLS?
Alias permissions	Can a researcher move a production alias? Is promotion a gated job?
Artifact credentials	Do clients hold bucket keys, or is access proxied?
Bucket safety	Versioning on? Write-restricted prefixes? Registry prefix exempt from expiry?
Backend	Postgres or MySQL, not SQLite? Disk and connection alerts?
Backups	Database and bucket? Restored in the last quarter? RTO published?
Pinning	Server image tag pinned? Client and server versions aligned? Base images by digest?
Retention	Scheduled, dry-run gated, exempting registry-referenced runs? Is <code>mlflow gc</code> running?
Logging discipline	Per epoch rather than per batch? No queries in loops?
Reproducibility	Dirty-tree guard? Dataset input required? Explicit serving requirements?
Rebuild check	Weekly, against the champion, with a tolerance, and currently green?
Lineage	Code, data, config, and approval for the champion in one command?
Promotion	Threshold-gated, relative to the baseline, with an audit tag and a reviewer?
Rollback	Is <code>previous</code> set at every promotion? Has the rollback been rehearsed?
Signatures	An additive-only policy? Validated in a built environment before promotion?
Serving	Authenticated? Resolves the alias at startup, not per request? Resource limits?
Supply chain	Who can write the artifact bucket? Are external models reviewed before loading?



Data exposure	<code>log_input_examples</code> on? Prediction artifacts classified? Trace retention decided?
Cost	Storage growth published per team? Lifecycle rules verified against the registry?
Boundaries	Documented where MLflow stops and what owns orchestration, data, and serving?

► TRY IT

1. Run the whole checklist against your own setup and count the failures.
2. Rank them by blast radius, not effort. Alias permissions and an untested restore usually top the list.
3. Fix the top three this week and re-run.
4. Then run it against a colleague's project, which is where you learn how much of this is convention rather than enforcement.

Expected result – nobody passes on the first attempt. The two that matter disproportionately are unrestricted alias permissions and an untested restore: one lets anyone change production, the other means a bad day becomes a permanent loss.



platform/mlflow_wrapper.py: conventions as defaults

Copy

```
import os, subprocess
import mlflow

IMAGE = "registry.internal/serving@sha256:9f2c4b19e0a7d3f1c6b8a2e4d7091f3b8a2e4d7091f3b"

def _git_clean() → bool:
    return subprocess.run(["git", "diff", "--quiet"]).returncode == 0

def start_run(project: str, purpose: str, name: str, promotable: bool = False, **kwargs):
    if promotable and not _git_clean():
        raise SystemExit("refusing to create a promotable run from a dirty tree")
    team = os.environ["ML_TEAM"]
    mlflow.set_experiment(f"{team}/{project}/{purpose}")
    tags = {"team": team, "cost_centre": os.environ["ML_COST_CENTRE"],
           "origin": os.environ.get("ML_ORIGIN", "manual"),
           "git_clean": str(_git_clean()), "base_image": IMAGE,
           **kwargs.pop("tags", {})}
    return mlflow.start_run(run_name=name, tags=tags, **kwargs)

def log_model(pyfunc_model, *, project: str, signature, dataset_logged: bool, **kwargs):
```



```
return mlflow.pyfunc.log_model(  
    name="model", python_model=pyfunc_model, signature=signature,  
    code_paths=["src/features.py", "src/model.py"],  
    pip_requirements="requirements-serving.txt",  
    registered_model_name=f"{team}-{project}", **kwargs,  
)
```

src/train.py: the researcher-facing surface stays small

Copy

```
import mlflow, mlflow.data  
from platform.mlflow_wrapper import start_run, log_model  
from src.model import ChurnModel  
  
SOURCE = f"s3://ml-data/churn/{PARTITION}/train.parquet"      # dated, immutable  
frame = read_parquet(SOURCE)  
dataset = mlflow.data.from_pandas(frame, source=SOURCE, name="churn-train", targets="lab  
  
with start_run("churn", "research", f"rf {PARTITION}", promotable=True) as run:  
    mlflow.log_input(dataset, context="training")  
    mlflow.log_params({"seed": SEED, "deterministic": True, "data_partition": PARTITION})  
    mlflow.sklearn.autolog(log_models=False, log_input_examples=False, silent=True)  
  
    model, signature = fit(frame, seed=SEED)  
    mlflow.log_metric("val_auc", model.val_auc)                # per epoch elsewhere  
    info = log_model(ChurnModel(), project="churn", signature=signature, dataset_logged=  
    print(info.model_uri)
```

the platform's scheduled work: all of it tracked runs

Copy

```
python platform/retention.py      # daily, dry-run gated, registry-aware  
python platform/rebuild_check.py  # weekly – reproducibility as a monitored property  
python platform/report.py         # weekly – runs and storage by team, alias inventory  
mlflow gc --backend-store-uri "$PG" --older-than 30d    # only this reclaims space
```

Eighteen decisions in there span the whole series:



A named experiment and a descriptive run name	Beginner
Params, metrics, tags, artifacts chosen correctly	Beginner
A signature on every logged model	Beginner
Consume by alias, never by version number	Beginner
<code>mlflow.evaluate</code> so every run is comparable	Beginner
An <code>MLproject</code> entry point, so a run is one command	Beginner
Autolog with <code>log_models=False</code> and no input examples	Mid
A custom pyfunc carrying preprocessing and the threshold	Mid
<code>code_paths</code> so the model imports nothing from the repo	Mid
An explicit, smaller serving requirements file	Mid
Thresholds relative to the champion, plus cost and segment metrics	Mid
A <code>previous</code> alias set at promotion, so rollback is one call	Mid
Auth and TLS, with proxied artifact access	Senior
Alias changes restricted to a gated, reviewed CI job	Senior
A dirty-tree guard and a required dataset input	Senior
Base image pinned by digest	Senior
Two-part backups, restored and timed	Senior
A weekly rebuild check, so reproducibility is monitored	Senior

► TRY IT: THE ONE THAT MATTERS

1. Write the wrapper for your own team and migrate one project onto it. Measure how much researcher-facing code disappeared.



4. Schedule the rebuild check and let it fail. Fix the unpinned layer it finds, then keep it green for a month.
5. Run the review checklist and publish the results, including what you are not going to fix and why.

Expected result – step five is the senior deliverable. A checklist with honest, argued exceptions beats a green one, because it shows the trade-offs were deliberate, and it gives whoever inherits the platform a map instead of a mystery.

Where the series leaves you

Across three levels you have gone from a handful of logging calls to owning a platform. You can track runs and structure sweeps; package arbitrary preprocessing into a portable model with a declared contract; register versions and promote by alias; gate promotions on thresholds relative to production; serve in batch, over HTTP, or in a container; and operate the whole thing with authentication, credential design, tested backups, monitored reproducibility, an audit chain, cost governance, and runbooks somebody else can follow.

The question a senior gets asked	The answer this series gives
"Can you reproduce the model we shipped in March?"	Clone its run: commit, params, dataset digest, image digest, and lock file are recorded, and the weekly rebuild check already proved it
"Who can put a model into production?"	Whoever can move an alias: the permissions, the gated workflow, and the audit tag
"What does this platform cost?"	Artifact storage and database growth, broken down by team and published weekly



"What breaks first if the team triples?"	The backend database, on metric write rate then run count, with a retention and capacity plan to match
"If the server dies, how long until we are back?"	A measured RTO from a restore we ran last quarter, and serving stays up, because it caches its model
"Prove this model is what we think it is"	One command producing code, data, configuration, and the timestamped approval
"Should we use MLflow for X?"	Where it is strongest, where it stops, and what owns the rest

The through-line of all three levels is a single idea: **a model is only useful if something other than its author can load it, and only trustworthy if something other than its author's memory explains it.** Everything else (signatures, aliases, pinned environments, audit tags, backups, rebuild checks) exists to keep that true as the number of models, people, and months grows.



PREVIOUS

Mid-level · Tips & Tricks

NEXT UP

Senior · Interview Review





LEARN

Roadmaps

Student Guides

Courses

The MLOps Practitioner

Sessions

Articles

COMMUNITY

Meet the team

Services for companies

FAQ

Contact us

Privacy Policy

WHERE WE ARE

WhatsApp

Discord

LinkedIn

X

YouTube

GitHub

Zomra



Brainsmingle space

Free to join this week with code **FREECORETEAM**



MLOps MENA
COMMUNITY

العربية



Join the Community ↗

